Development of an autonomous agent for the game **Arkanoid**

Group Assignment

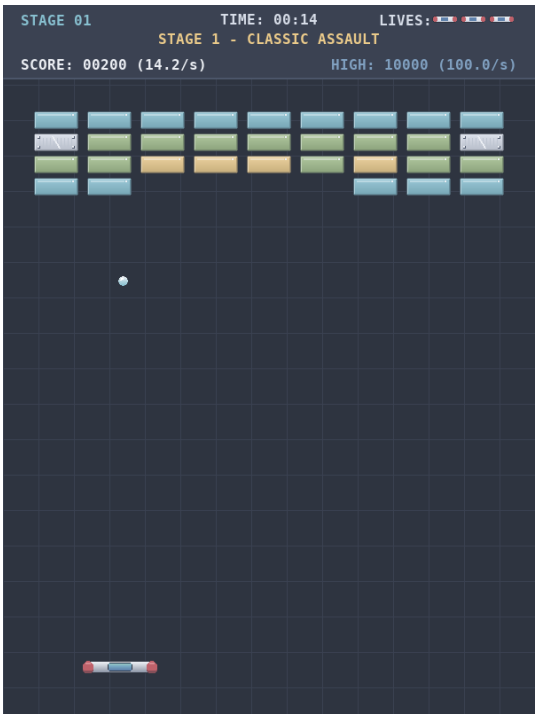
Artificial Intelligence

Academic Year 2026/2027

Mário Antunes
mario.antunes@ua.pt

Luís Seabra Lopes
lsl@ua.pt

October 2026



I Important notes

1. This work must be carried out in groups of three students. Group members must be identified by name and mechanographic number in the `README.md` of their repository.

2. A first version of the program must be submitted by November 18, 2026. The final version must be submitted by December 16, 2026. In both submissions, the work can be submitted beyond the deadline, but will be penalized in 5% for each additional day.

3. Each group must submit its code in *GitHub* (the repository of the group). In the final submission, include a presentation (Powerpoint type) in `.pdf` format and named `presentation.pdf`, with a maximum of five pages, where you should summarize the architecture of the developed agent.

4. The code should be developed in Python 3.12, the default version used to run and evaluate the agents (3.10 is the minimum supported). The main module should be named `student.py`. The agent is evaluated in *inference mode* and can only rely on the libraries declared in `pyproject.toml`: any other library (including those used to train a model) is not installed, so the trained model must be stored in files of the repository and run with those libraries.

5. If you discuss this work with colleagues from other groups, include a comment with the name and mechanographic number of these colleagues. If you use other sources, cite them as well.

6. All code submitted must be original; though trusting that most groups will comply with this requirement, tools will be used to detect plagiarism. Students involved in plagiarism cases will have the assignment canceled.

7. The project will be evaluated taking into account: performance; quality of architecture and implementation; and originality.

II Overview

This work involves the application of concepts and techniques from the main chapters of the AI course, namely: Python programming; agent architectures; and techniques for automated decision making and problem solving.

Within the scope of this work, you should develop an agent capable of playing intelligently a variant of the game Arkanoid, an arcade game developed by Taito in Japan in 1986.

In *Arkanoid*, the player controls a paddle (the *Vaus*) that slides horizontally along the bottom of the screen. A ball bounces around the playfield and the player must keep it in play by hitting it with the paddle. Each time the ball touches a brick, the brick is damaged and, when its health reaches zero, it breaks and awards points. A level is completed when all destructible bricks have been broken.

The main objective is to clear all levels while accumulating points and without losing the ball. Bricks have different resistance: the more hits a brick needs, the more points it is worth. Some bricks are made of metal, cannot be destroyed and only deflect the ball. Missing the ball costs one life; the game ends when all lives are lost or when all levels are cleared.

This variant is deliberately simple to observe and hard to master: the agent controls only the horizontal position of the paddle, but where the ball hits the paddle (left, center or right third) decides where the ball goes next. A good agent therefore does not only *follow* the ball, it also *aims* it.

III Game Rules

- The playfield is 600×800 pixels, with a ceiling at $y = 82$ (the area above is the HUD). Bricks are 50×20 pixels. The paddle is 84×14 pixels and the ball has a radius of 6 pixels.
- The *Arkanoid* player (student agent) has access at all times to the entire state: paddle, ball position and velocity, all active bricks, lives, score and time. The ball moves at a constant speed of 400 pixels per second, with continuous collision detection (it cannot tunnel through bricks).
- The game runs at 30 ticks per second of real time. At each tick the server sends the state and the agent answers with an action: `{"action": "move", "direction": "WEST"}` or `{"action": "move", "direction": "EAST"}`. Each action moves the paddle by 28 pixels as soon as it is received; no answer means the paddle stays still. The server does not wait for the agent, so a slow agent simply reacts to fewer states.
- Bricks have a color that tells how many hits they need: red (1 hit, 50 points), blue (2 hits, 100 points), green (3 hits, 150 points) and yellow (4 hits, 200 points). Points are awarded *only when a brick breaks*; damaging a brick scores nothing. Metal bricks are indestructible, score nothing and just deflect the ball.
- The paddle is divided into three equal zones that set the bounce angle: the **center** third sends the ball almost straight up (a small random cone around 90°); the **left** third sends it to the left at about 45° and the **right** third to the right at about 45°, each with a small random spread. The ball speed never changes, only its direction.
- The game is composed of 3 levels of increasing difficulty, played in sequence (the first has a classic wall, the last has metal columns and 4-hit yellow bricks). A level ends as soon as no destructible brick is left, even if metal bricks remain. The ball is relaunched from the paddle, with a slight random slant, at the start of each level and after each lost life.
- The agent has 3 lives per game. A game is over when the third life is lost, and is won when all the levels played are cleared. Each level has a maximum duration of 400 seconds of game time (simulated time, so it does not change with the server `--acceleration`); the clock restarts at every level. The game only moves on to the next level if the current one is *cleared*; if the maximum duration is reached first, the game is over (`timed_out`) with the score and time accumulated up to that moment. The levels to play are selected when the server is started (parameter `--levels`, e.g. `--levels 2` plays only level 2), and the maximum duration can be changed with `--level-time-limit`.
- The raw `score` is the sum of the points of all the levels (the high score). It ranks the agents that get the same performance value, and it is all that an agent that clears no level has.
- The `normalized_score` is a per-level metric: the points obtained in the level divided by the seconds of game time of that level (the clock restarts at every level). It only counts if the level is *cleared*: a level that is lost or times out earns no normalized score (0).
- The performance measure is the `avg_norm_score`: the mean of the `normalized_score` over all the levels of the sequence played, counting 0 for the levels that were not cleared or not reached. It is the value sent to the server and used for the marking, and it is shown in the HUD.

IV Code and Development Support

Your repository contains the server (`server/`), a Pygame viewer (`viewer/`), example agents (`agents/`), the level files (`maps/`) and the game constants (`config.json`). The `README.md` explains how to create the virtual environment, how to install the requirements (they are declared in `pyproject.toml`, there is no `requirements.txt`: run `pip install .` inside the active virtual environment) and how to start the server, the viewer and an agent.

Each group develops an agent creating a client that follows the protocol exemplified in `agents/dummy_agent.py`: subclass `BaseAgent` and implement `deliberate()`, which reads `self.current_state` and returns an action (or `None`). `agents/manual_agent.py` lets you play with the keyboard to get a feeling for the game. No modifications to the server files are necessary (you can't change `server/logic.py`), but you can create new files, folders, etc.

The state sent to the agent is a JSON object, from which the following fields are the most useful:

`paddle_x`, `paddle_width`	paddle position (left edge) and size
`ball_x`, `ball_y`, `ball_vx`, `ball_vy`	ball center and velocity (pixels, pixels/s)
`bricks`	active bricks: `x`, `y`, `type`, `hits_remaining`, `points`, `indestructible`
`lives`, `score`, `normalized_score`, `avg_norm_score`, `time_elapsed`	game progress
`stage`, `game_over`, `game_won`, `timed_out`	level and end of game
`level_time_elapsed`, `level_time_limit`	time used and allowed in the current level
`valid_actions`	moves allowed in this tick

If you implement a new feature or implement any improvement to the game engine and/or viewer, you can create a "Pull Request" (PR) on the GitHub platform. If your change is accepted, you will be credited with a bonus on the final evaluation up to a maximum of 1 point (in 20).

The developed agent must be delivered in a module named `student.py` and the agent should connect to the local game server, using as *id* (the `--id` argument, or the `AGENT_ID` variable of the module) the mechanographic number of one of the elements of the group (any). The *id* is sent to the server and is what identifies and ranks the group in the dashboard; the `--name` argument is only a display name. If no *id* is given, a random one is generated, so a submission without *id* cannot be ranked. The `--id` argument takes precedence over `AGENT_ID`.

There is a support channel in `https://detiuaveiro.slack.com/messages/ai/` where students can ask questions and receive notifications of changes.

Given the novelty of the game engine, it is expected that there are some bugs and tweaks during the course of work. Also, a *dashboard* with the ranking of the agents will be released later. Therefore, several updates of the template repository are expected: bug fixes, new levels and the dashboard itself. Be on the watch out for these modifications (configure the template repository as an upstream repository and fetch/merge often) and notifications in *e-mail*, *Slack* and *e-learning*.

V Evaluation

Agent performance will be evaluated on a *sequence of maps*: the three levels provided in the repository plus some additional maps that are not published. Each game goes through the maps of the sequence in order (with 3 lives for the whole game), and moves on to the next map only

if the current one is cleared within its maximum duration of 400 seconds. The value of a game is its `avg_norm_score`: the mean, over all the maps of the sequence, of the points per second obtained in each map, counting 0 for the maps that were not cleared and for the maps that were not reached. The game ends when the agent loses its last life, when the current map is not cleared within its maximum duration, or when all the maps are cleared. An agent that clears no map gets 0, however many points it collects; the raw score is only used to break ties. The values of all the games of an agent are averaged into a single value, so a weak map lowers the final result.

The average `avg_norm_score` of each submitted agent is mapped to marks taking into account the distribution of these averages. A value of 10 points per second is mapped to 10/20, the median to 16/20 and the maximum to 20/20; other values are mapped linearly (0 is mapped to 0/20). There is no other minimum: the weights below are applied to the performance mark.

1st delivery evaluation

- In this delivery, only the agent code must be submitted.
- Each agent will play 10 games of the sequence of maps and the average of these games will be obtained.

2nd delivery evaluation

- Each agent will play 10 games of the sequence of maps.
- In this delivery it is necessary to submit a presentation (see point I.3 above).
- The evaluation of the second delivery reflects the agent's performance (90%), the design quality (according to the delivered presentation, 7.5%) and the quality of the implementation (2.5%).

VI Recommendations

1. Start by reading `README.md` and following its quick start *in order*: create the virtual environment, activate it, *then* install the requirements with `pip install .`. Activating a virtual environment does not install anything, and the framework (`ai-game-framework`) is not installed by hand.
2. Then study `agents/dummy_agent.py` and `agents/base_agent.py`. The code is very basic and simple, so start by *refactoring* the client to something more oriented to an autonomous AI Agent.
3. Start with an agent that simply tracks the ball, and measure it. Then improve it: use the velocity to predict where the ball will land (including wall bounces), and choose which third of the paddle to use to send the ball where it breaks the most valuable bricks.
4. Use `--acceleration` and `--levels` to run the server faster, or only on one level, while you develop and compare versions (see the README). Compare versions with several games: the launch angle is random.
5. Periodically `git fetch` from the template repository to update your code.
6. Run `git log` to keep track of small changes that have been made.
7. Follow the channel #ai on Slack

VII Clarification of doubts

Clarifications on the main doubts that may arise during the performance of the work will be placed here.

1. **Question**: The `pip install` of the framework fails with "Could not find a version that satisfies the requirement". What is wrong?

   Answer: The framework is the PyPI package `ai-game-framework` (imported as `aigf`); there is no package called `aigf`. Do not install packages one by one: activate the virtual environment and run `pip install .` from the repository root, which installs everything declared in `pyproject.toml`.